



Snow Crash

A security project

42 Staff pedago@42.fr

Summary: This project is an introduction to cyber security.

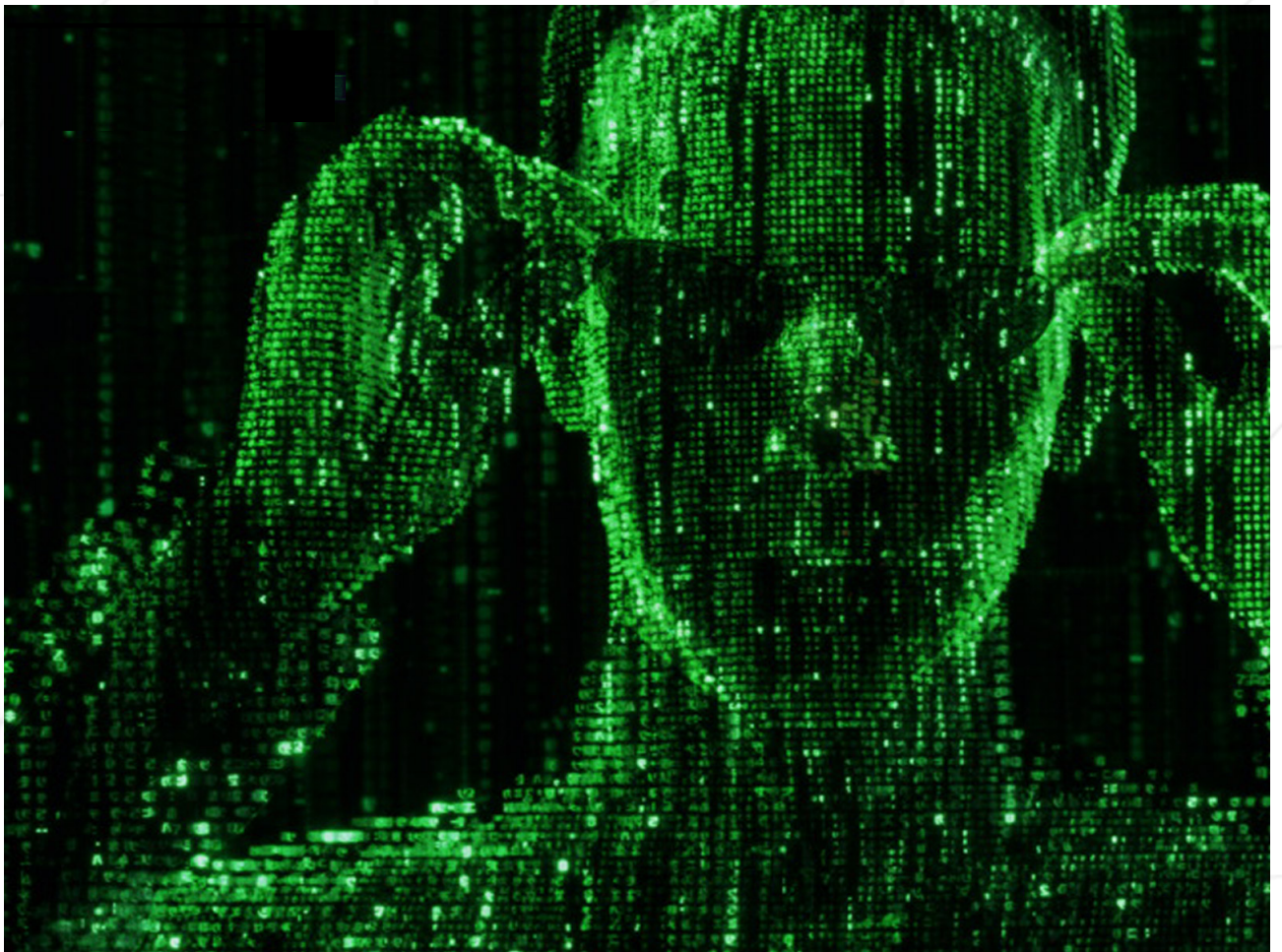
Version: 4.0

Contents

I	Foreword	2
II	Introduction	3
III	Objectives	4
IV	General instructions	5
V	How progression works	7
VI	Mandatory part	9
VI.1	Boss fight: level10	10
VII	Bonus part	11
VIII	Submission and peer-evaluation	12

Chapter I

Foreword



Chapter II

Introduction

As a developer, you will work on software used by many people. Every line you write runs on someone else's machine, handles someone else's data, and trusts input you did not control.

If your software has weaknesses, it will expose its users. Understanding how those weaknesses are found and abused is not optional. It is the only reliable way to recognize them in your own code, fix them, and avoid writing them in the first place.

Snow Crash drops you into a deliberately vulnerable machine and asks you to break it. You begin as `level100` and climb one account at a time. Each level hides a flag that only the next account is allowed to read, and that flag is the password that unlocks the following stage. To progress, you have to understand the program or service in front of you well enough to turn its own behavior against it.

Be warned: this project is demanding from the very first level. These are not puzzles with a trick to guess. They reward patient reading, careful analysis, and a stubborn refusal to move on until you truly understand what you are looking at. A few levels fall quickly once the idea clicks. Most do not. The bonus levels go much further still and are considerably harder than anything in the mandatory part.

There are no hints inside the machine. Read the system, use your tools, and let what you observe decide what you try next.



This project is deliberately complex. Getting stuck is part of the experience, not a sign of failure. Take your time and do not give up.

Chapter III

Objectives

11 levels. One VM. Each level teaches a different facet of systems security through a real exploit rather than a textbook exercise.

By the end you will be comfortable reading unfamiliar programs and services, reasoning about how they handle their input and their privileges, and spotting the gap between what a system was meant to do and what it can actually be made to do. You will move across several languages and technologies and rely on the inspection and debugging tools already present on the machine. Working out which tool fits each situation is part of the challenge.

You may reach dead ends. Step back, re-read what you have, and look at the problem from a different angle. The answer is always in the system: you just have to find it.



This project is done in a group. Different levels naturally suit different skill sets. Share what you find and work together.

Chapter IV

General instructions

- This project will be evaluated by humans.
- You may need to demonstrate your results during your evaluation. Be ready to walk through every step.
- To complete this project, import the provided .ova appliance into VirtualBox and start it. Once the VM has finished booting, its console will look like this:

```
Ubuntu 24.04.4 LTS snowcrash tty1
snowcrash login: level100
Password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-124-generic x86_64)

System information as of Tue Jun  9 09:09:50 PM UTC 2026

System load:  0.0          Processes:      141
Usage of /:   47.3% of 9.74GB Users logged in:  0
Memory usage: 9%          IPv4 address for enp0s3: 10.0.2.15
Swap usage:   0%

SnowCrash

42 Network Security Challenge - v2  @wil
level100@snowcrash:~$ _
```

- Connect via SSH on port **4242** with the entry credentials: username level100 and password level100.

```
$> ssh level100@<vm-ip> -p 4242
level100@<vm-ip>'s password: level100
level100@snowcrash:~$
```

- From your host machine, the server is reachable on the network. Identifying its exact address is straightforward with VirtualBox NAT.



You cannot modify the .ova file or create an altered copy under any circumstances. You must exploit the **running server**.

How progression works

- ```
level00@snowcrash:~$ <exploit>
<flag string>
level00@snowcrash:~$ exit
```

```
$> ssh level01@<vm-ip> -p 4242
Password: <flag string>
level01@snowcrash:~$
```

```
$> ssh level00@10.0.2.15 -p 4242
level00@10.0.2.15's password:
```

# Stylish Grid

42 Network Security Challenge – v2 @wil

```
level00@snowcrash:~$ ls
```

# README

```
level00@snowcrash:~$ cat README
```

Hiro Protagonist. Last of the freelance hackers. Or so he thinks.

```
level00@snowcrash:~$ id
```

```
uid=1001(level00) gid=1003(level00) groups=1003(level00),1001(levelgroup)
```

```
level00@snowcrash:~$
```



- You cannot brute-force the SSH login. You must find the flag through the intended exploit path and be able to explain every step.



Brute-forcing SSH is strictly forbidden and will result in an immediate failure of the evaluation.

# Chapter VI

## Mandatory part

- For the mandatory part, you must complete **level00 through level10** (11 levels, including the boss fight).
- Your repository must include everything needed to prove and explain your solution for each level. Use one folder per level:

```
$> ls
level00 level01 level02 level03 level04
level05 level06 level07 level08 level09 level10

$> ls level00/
flag resources/

$> cat level00/flag | cat -e
XXXXXXXXXXXXXXXXXXXXXXXXXXXXX$
```

- The **flag** file contains the flag you recovered. The **resources/** folder contains any scripts or files you used to solve the level.
- You must be able to **clearly explain every step** of your solution during the evaluation.



Your repository must not contain any binary files. If you need a file from the VM, download it during the evaluation.

- If you use external tools, set up a proper environment (VM, Docker, Vagrant).
- Scripts that automate your solution are welcome, but you must be able to explain them in detail.

## VI.1 Boss fight: level10

Once you have completed `level09`, the flag you recover becomes the SSH password for `level10`. This is the final challenge of the mandatory path.

`level10` chains two technologies together to reach the flag. The difficulty is higher than in all previous levels. It requires you to understand, reconstruct, and manipulate a non-trivial binary protocol before you can inject anything.

Include a `level10/` folder in your repository, with the same structure as the other mandatory levels (`flag` + `resources/`).



`level10` is part of the mandatory path. It will be assessed like any other mandatory level.

# Chapter VII

## Bonus part



Bonus will only be assessed if the mandatory part is **PERFECT**, including level10. If any level is incomplete or unexplained, bonus levels will not be evaluated.

Beyond the 11 mandatory levels, the server hosts **5 additional challenges** (bonus01 through bonus05). Each one uses a distinct technique that appears nowhere in the mandatory path.

Be warned: the bonus levels are **significantly harder** than the mandatory ones. They assume you are already fluent in everything the main chain taught you, then push well past it. Do not expect them to fall quickly, and do not start them until your mandatory part is solid.

For each bonus level you complete, include a **bonusNN/** folder in your repository with the same structure as the mandatory levels: a **flag** file and a **resources/** folder.



For starters, since we are on the bonus part... yeah, you will find that the first password for the first user isn't the last flag. I'm sure that you will find it! Good luck :)

# Chapter VIII

## Submission and peer-evaluation

Submit your assignment to your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense.

How you organize your **resources**/ folders is entirely up to you, as long as your work is easy to navigate and you can explain all of it during the evaluation. Double-check the names of your folders and files to ensure they are correct.